

Biological validation of co-phased H1 chromatin contact maps - comparison of chromatin compartment status and their expression level

Julia Markowki

3/2/2021

Introduction

With our Co-Phasing approach we produced phased chromatin contact maps of H1 (I believe 200kb resolution, 200kb max distance between SNV and read), of which Sasha called compartments using PC1. Here we want to validate the results biologically by comparing chromatin compartments to activity aka expression. Teresa provided preprocessed publicly available GRO-Seq data for H1, showing nascent transcription activity, where expression values show read counts per genomic window in either 50kb or 100kb resolution; or Poly-A enriched RNA-Seq of H1 per gene. Expression data is not phased. The aim is to show that regions in which both homologous chromosomes are “active” (compartment A) show the highest expression, differentially called compartments, where one chromosome copy is “active” (A) and the other one “inactive” (B), show moderate, but at least lower expression, and regions where both homologous chromosomes are “inactive” (B) show the lowest expression. Here I want to demonstrate those differences and show, that they are significant.

Data

In the first step I will load and clean the data, and explore the compartment calls and expression values independently. Later I will merge this information and proceed with the validation.

Compartments

Sasha called chromatin compartments using PC1 on the two co-phased H1 chromatin contact maps in (presumably) 200kb resolution, as this yields robust results. Now the compartments can either be both active A, one active one inactive AB, or both inactive B. The Genome Scores describe Eigenvalues.

Preprocessing

```
### load compartments -----
comp=read.csv("/fast/groups/ag_schwarz/Projects/project-gam/data/Co-Phasing/compartments/210222.Compartments.AB.H1.phased.first.
comp=read.csv("/Users/jmarkow/Desktop/GAM/Co-Phasing/data/compartments/210222.Compartments.AB.H1.phased.first.PCA.csv",header =

### preprocess -----
# check size
dim(comp) # 11749      7

## [1] 11749      7
# sort by chromosome and position
comp = comp[order(comp$start),]
comp = comp[order(as.numeric(substring(comp$chrom, 4))),]

# add 1bp to every start position so that intervals are exclusive and not overlapping
comp$start = comp$start + 1

# check chromosome set
head(comp)

##      chrom      start      end Genome1_AB1 Genome1_score1 Genome2_AB1 Genome2_score1
```

```
## 10409 chr1 1600001 1800000 B -2.9085194 B -2.94957055
## 10079 chr1 1800001 2000000 B -2.1595080 A 0.44655609
## 3121 chr1 2000001 2200000 B -2.1920735 A 0.43363692
## 3710 chr1 3200001 3400000 B -0.7106669 B -0.03870505
## 4778 chr1 3800001 4000000 B -1.0402007 A 0.96913599
## 415 chr1 4000001 4200000 A 1.9779379 A 1.10438412
```

```
unique(comp$chrom)
```

```
## [1] chr1 chr2 chr3 chr4 chr5 chr6 chr7 chr8 chr9 chr10 chr11 chr12 chr13 chr14 chr15 chr16 chr17 chr18 chr19 chr20
## Levels: chr1 chr10 chr11 chr12 chr13 chr14 chr15 chr16 chr17 chr18 chr19 chr2 chr20 chr21 chr3 chr4 chr5 chr6 chr7 chr8 chr9
# no need to exclude sex or mitochondria chromosomes
```

```
# combine called compartments of both chromosomes into one value
comp$combined_comp_raw = paste(comp$Genome1_AB1, comp$Genome2_AB1, sep = '')
comp$combined_comp_raw = factor(comp$combined_comp_raw, levels = c("AA", "AB", "BA", "BB"))
```

```
# table(comp$combined_comp_raw) / nrow(comp) * 100
#      AA      AB      BA      BB
# 32.75172 18.59733 21.46566 27.18529
```

```
# reduce the compartment calls to AA / AB / BB
comp$combined_comp = comp$combined_comp_raw
comp$combined_comp[which(comp$combined_comp == "BA")] = "AB"
comp$combined_comp = factor(comp$combined_comp, levels = c("AA", "AB", "BB"))
```

```
# how many genomic windows are differentially called?
comp_dist = table(comp$combined_comp) / nrow(comp) * 100
comp_dist
```

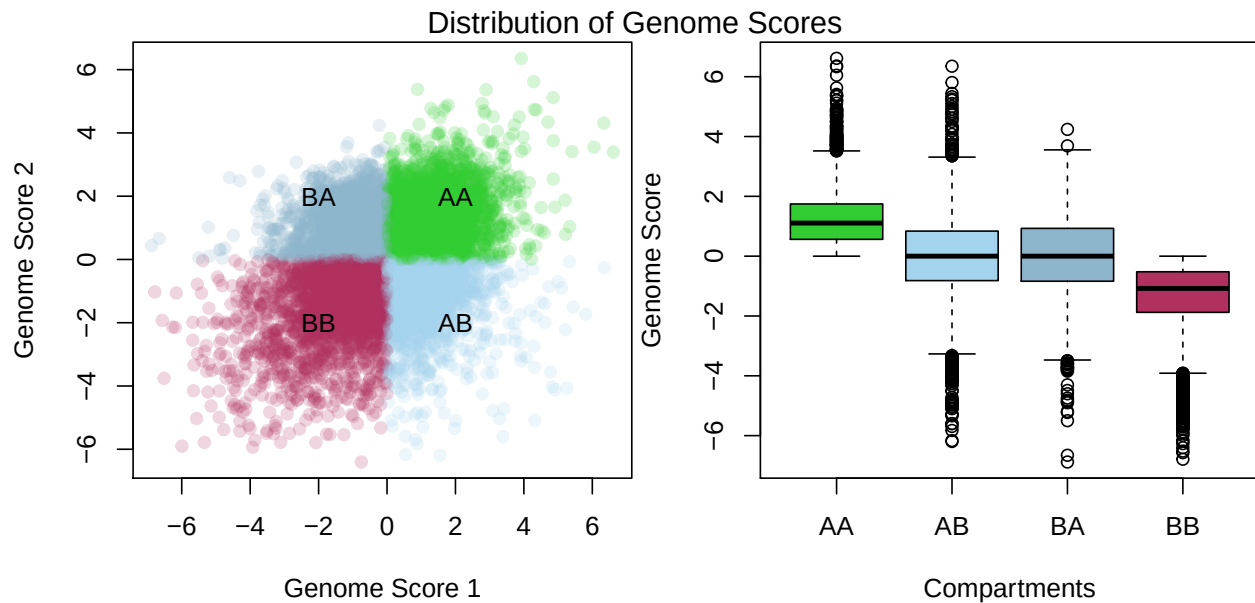
```
##
##      AA      AB      BB
## 32.75172 40.06298 27.18529
```

The compartment dataset only includes autosomes. 59.9370159 % of compartments agree between the homologous chromosomes, 40.0629841 % of compartments differ.

Distribution of Genome Scores

```
par(mfrow=c(1,2),oma = c(0, 0, 2, 0),par(mar=c(4,4,0.1,0.1))) #bltr margins
```

```
plot(x = comp$Genome1_score1,
     y = comp$Genome2_score1,
     pch = 19,
     col = adjustcolor(c("limegreen", "lightskyblue2", "lightskyblue3", "maroon")[comp$combined_comp_raw], alpha.f=0.2),
     xlab = "Genome Score 1",
     ylab = "Genome Score 2")
text(x=2,y=2,labels = c("AA"))
text(x=2,y=-2,labels = c("AB"))
text(x=-2,y=2,labels = c("BA"))
text(x=-2,y=-2,labels = c("BB"))
boxplot(c(comp$Genome2_score1, comp$Genome1_score1) ~ c(comp$combined_comp_raw, comp$combined_comp_raw), outline=T, xaxt='none', col=c("AA", "AB", "BA", "BB"))
axis(1, at=1:4, label=c("AA", "AB", "BA", "BB"))
mtext("Distribution of Genome Scores", outer = TRUE, cex = 1.2)
```



Expression

Teresa used publicly available gene expression (poly-A enriched RNA-Seq) and nascent expression (GRO-Seq) data and provided expression values either per gene or in 50kb and 100kb windows, respectively. The values show read counts per window. We will concentrate on nascent expression data first and analyse the gene expression data later on.

Preprocessing

```
### load expression data -----
gene_exp=read.csv("/Users/jmarkow/Desktop/GAM/Co-Phasing/data/expression_data/genes_with_expression.bed",header = T, sep="\t")
gro_50=read.csv("/Users/jmarkow/Desktop/GAM/Co-Phasing/data/expression_data/groseq_intersected_50kb.bed",header = F, sep="\t")
gro_100=read.csv("/Users/jmarkow/Desktop/GAM/Co-Phasing/data/expression_data/groseq_intersected_100kb.bed",header = F, sep="\t")

### concentrate on GRO-Seq first -----
colnames(gro_50)=c("chrom", "start", "end", "expression")
colnames(gro_100)=c("chrom", "start", "end", "expression")

### preprocess -----
# check size
dim(gro_50) # 60630      4

## [1] 60630      4
dim(gro_100) # 30321      4

## [1] 30321      4
# sort by chromosome and position
gro_50 = gro_50[order(gro_50$start),]
gro_50 = gro_50[order(as.numeric(substring(gro_50$chrom, 4))),]

## Warning in order(as.numeric(substring(gro_50$chrom, 4))): NAs introduced by coercion
gro_100 = gro_100[order(gro_100$start),]
gro_100 = gro_100[order(as.numeric(substring(gro_100$chrom, 4))),]

## Warning in order(as.numeric(substring(gro_100$chrom, 4))): NAs introduced by coercion
# add 1 to every start position so that intervals are exclusive
gro_50$start = gro_50$start + 1
gro_100$start = gro_100$start + 1
```

```

# check chromosome set
unique(gro_50$chrom) # unique(gro_100$chrom)

## [1] chr1 chr2 chr3 chr4 chr5 chr6 chr7 chr8 chr9 chr10 chr11 chr12 chr13 chr14 chr15 chr16 chr17 chr18 chr19 chr20
## Levels: chr1 chr10 chr11 chr12 chr13 chr14 chr15 chr16 chr17 chr18 chr19 chr2 chr20 chr21 chr22 chr3 chr4 chr5 chr6 chr7 chr8
# "chr1" "chr2" "chr3" "chr4" "chr5" "chr6" "chr7" "chr8" "chr9" "chr10" "chr11" "chr12" "chr13" "chr14" "chr15" "c

# remove chr X
gro_50=gro_50[which(gro_50$chrom!="chrX"),]
dim(gro_50) # 57509 4

## [1] 57509 4

gro_100=gro_100[which(gro_100$chrom!="chrX"),]
dim(gro_100) # 28760 4

## [1] 28760 4

```

Distribution of Expression data

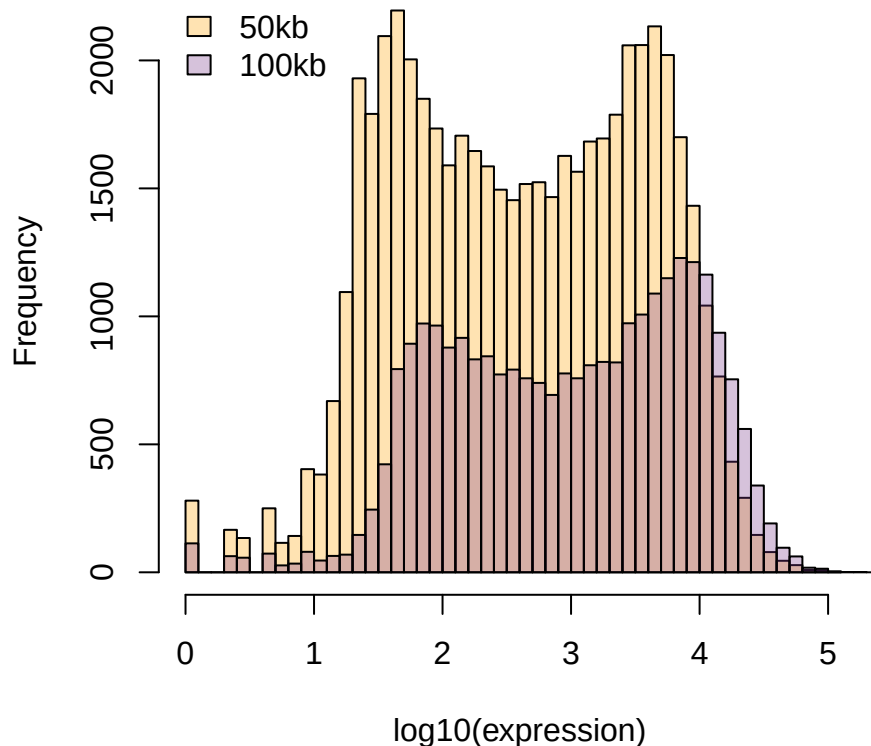
Do the distributions of 50kb and 100 kb windows overlap?

```

hist(log10(gro_50$expression),
     breaks=c(seq(0,5.1,0.1)),
     main="histogramm of nascent expression values per bin",
     xlab="log10(expression)",
     xlim=c(0,max(max(log10(gro_50$expression)),max(log10(gro_100$expression)))),
     col=adjustcolor("orange",alpha.f = 0.3))
hist(log10(gro_100$expression),
     breaks=c(seq(0,6.1,0.1)),
     col=adjustcolor("mediumorchid4",alpha.f = 0.3),
     add=T)
legend("topleft",fill=c(adjustcolor("orange",alpha.f = 0.3),adjustcolor("mediumorchid4",alpha.f = 0.3)),legend=c("50kb","100kb"))

```

histogramm of nascent expression values per bin



```

### test if this holds statistically
# divide 100kb values in half, double them in size, see if distributions fit
# (they must deviate at some point a bit, but should be the same distribution)
halved100=rep(gro_100$expression/2,times=2)
halved100[which(halved100>0 & halved100<1)]=1

t.test(gro_50$expression,gro_100$expression)

##
## Welch Two Sample t-test
##
## data: gro_50$expression and gro_100$expression
## t = -43.878, df = 38655, p-value < 2.2e-16
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
## -2271.589 -2077.323
## sample estimates:
## mean of x mean of y
## 2175.253 4349.709

# As expected, the 50kb and 100kb data differ significantly, but let's compare fair:

t.test(gro_50$expression,halved100)

##
## Welch Two Sample t-test
##
## data: gro_50$expression and halved100
## t = 0.015994, df = 112650, p-value = 0.9872
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
## -48.21746 49.01085
## sample estimates:
## mean of x mean of y
## 2175.253 2174.856

# The distributions do not differ.

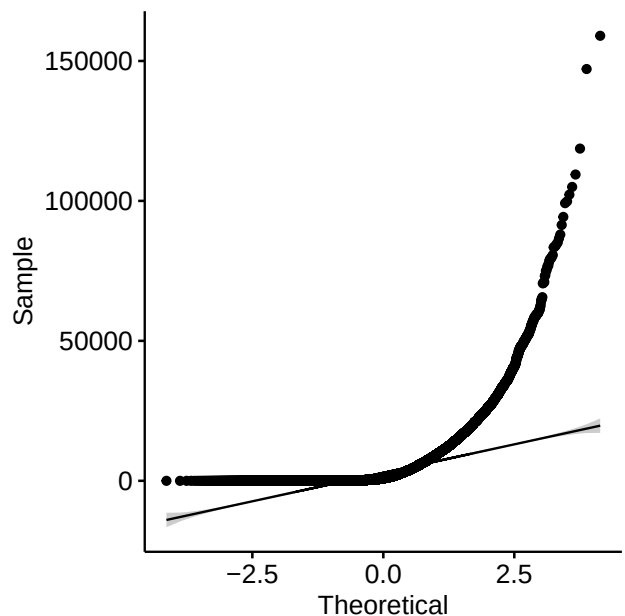
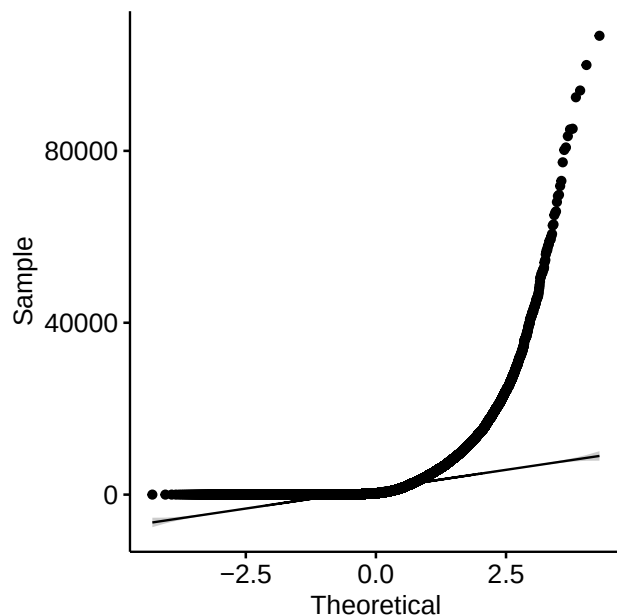
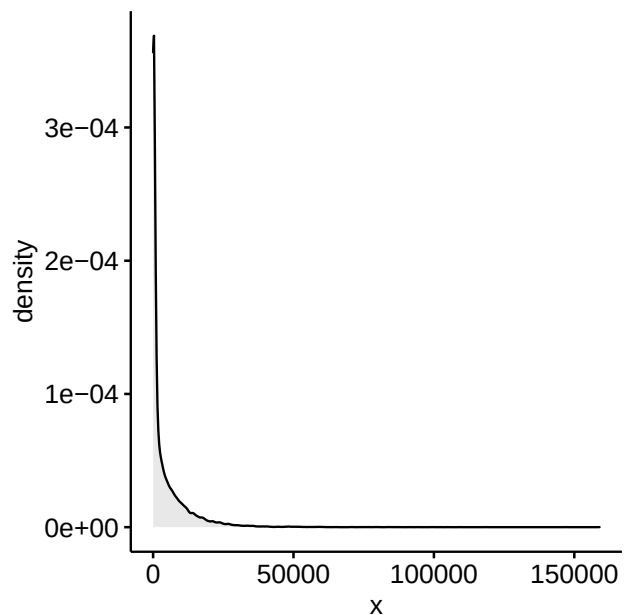
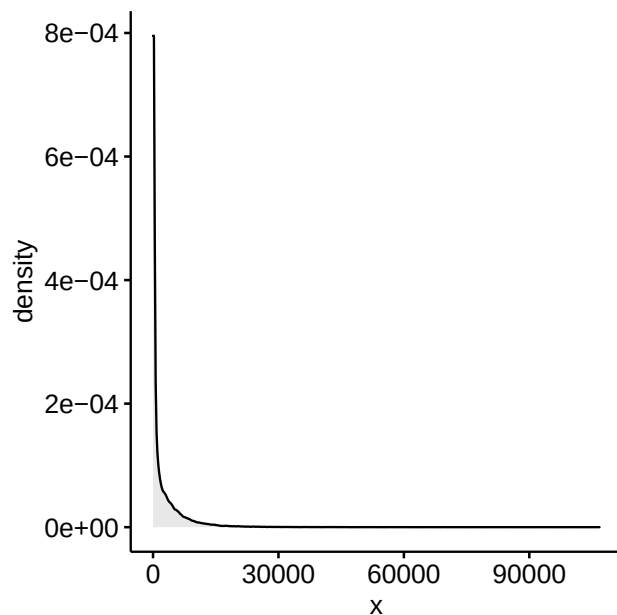
```

Yes, they do overlap and do not significantly differ. This was expected as both the 50kb values as well as 100kb values were derived from the same data. The distribution looks bimodal. Does the expression data follow a normal distribution?

```

library("rstatix")
require("gridExtra")
library("ggplot2")
library("ggpubr")
### check if normal distribution criterium is met -----
# visual inspection
# density plot
p1 =ggdensity(gro_50$expression, fill = "lightgray")
p2 =ggdensity(gro_100$expression, fill = "lightgray")
# qqplot
p3 =ggqqplot(gro_50$expression)
p4 =ggqqplot(gro_100$expression)
grid.arrange(p1, p2, p3, p4, ncol=2)

```



```
# visual inspection suggests no normal distribution,
# both resolutions don't really look like it, let's test it
```

```
# There are several methods to evaluate normality, including the Kolmogorov-Smirnov
# normality test and the Shapiro-Wilk's test. The null hypothesis of these tests
# is that the "sample distribution is normal".
# If the test is significant, the distribution is non-normal.
# Shapiro-Wilk's method is widely recommended for normality test and it provides
# better power than K-S. It is based on the correlation between the data and the
# corresponding normal scores (Ghasemi and Zahediasl 2012).
```

```
# comp_gro100_red %>% shapiro_test(expression)
# Error: Problem with `mutate()` input `data`.
# x sample size must be between 3 and 5000
# nrow(comp_gro100_red) # 23498
# table(comp_gro100_red$combined_comp)
#   AA  AB  BB
# 7696 9414 6388
```

```

# not even every single group has less than 5000 entries
# ok I need to sample from the whole
set.seed(1)
reduced1=sample(gro_50$expression,5000,replace=F)
shapiro.test(reduced1)

##
## Shapiro-Wilk normality test
##
## data: reduced1
## W = 0.50388, p-value < 2.2e-16
set.seed(5000)
reduced2=sample(gro_50$expression,5000,replace=F)
shapiro.test(reduced2)

##
## Shapiro-Wilk normality test
##
## data: reduced2
## W = 0.47443, p-value < 2.2e-16
set.seed(2992)
reduced3=sample(gro_50$expression,5000,replace=F)
shapiro.test(reduced3)

##
## Shapiro-Wilk normality test
##
## data: reduced3
## W = 0.52863, p-value < 2.2e-16
set.seed(1)
reduced1=sample(gro_100$expression,5000,replace=F)
shapiro.test(reduced1)

##
## Shapiro-Wilk normality test
##
## data: reduced1
## W = 0.58337, p-value < 2.2e-16
set.seed(5000)
reduced2=sample(gro_100$expression,5000,replace=F)
shapiro.test(reduced2)

##
## Shapiro-Wilk normality test
##
## data: reduced2
## W = 0.56007, p-value < 2.2e-16
set.seed(2992)
reduced3=sample(gro_100$expression,5000,replace=F)
shapiro.test(reduced3)

##
## Shapiro-Wilk normality test
##
## data: reduced3
## W = 0.61397, p-value < 2.2e-16
# tried both sets 3 times, all times with highly significant results,
# this data not normally distributed (in line with the visual inspection)

```

No, the expression data is not normally distributed.

Combine compartment data with nascent expression data

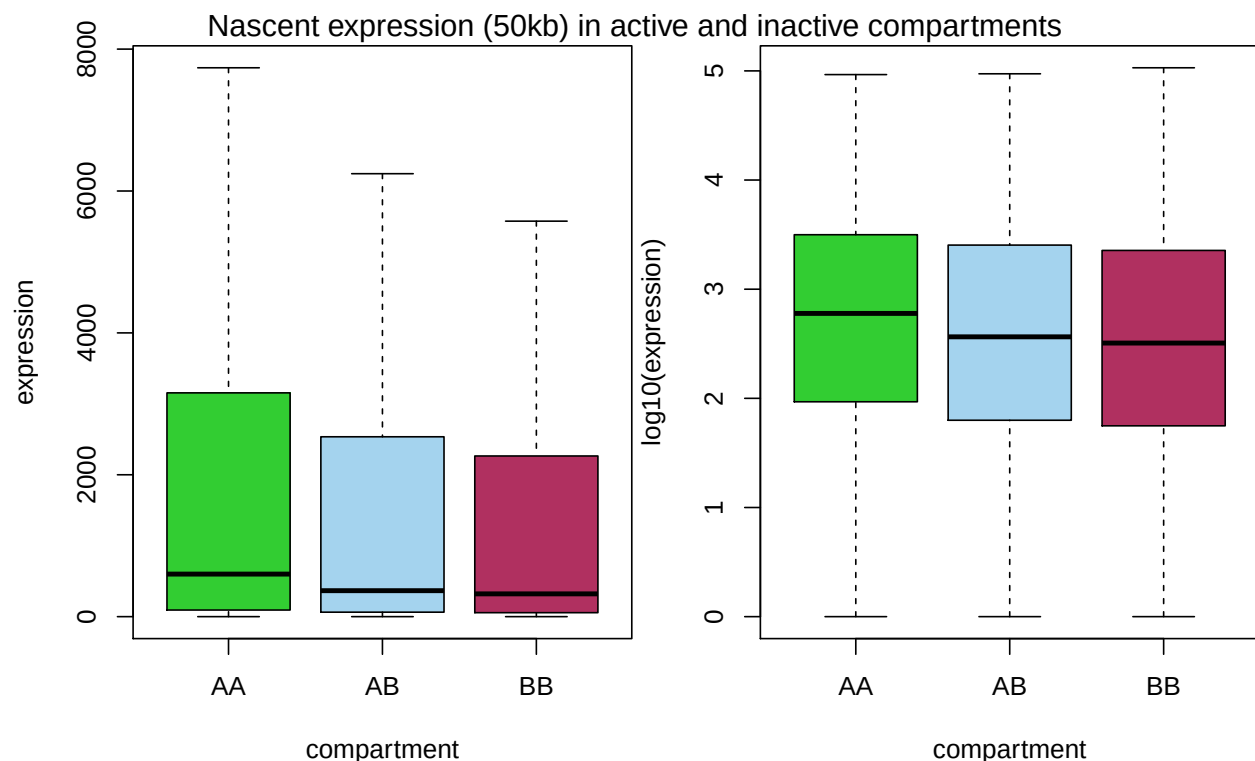
Let's see if the expression differs between the called compartment levels "AA", "AB" and "BB". First, combine the datasets.

```
require(data.table)
comp_dt=as.data.table(comp)
gro_50_dt=as.data.table(gro_50)
setkey(gro_50_dt, chrom, start, end)
comp_gro50=foverlaps(comp_dt, gro_50_dt, type="any", mult="all") # , minoverlap=2 not yet implemented

gro_100_dt=as.data.table(gro_100)
setkey(gro_100_dt, chrom, start, end)
comp_gro100=foverlaps(comp_dt, gro_100_dt, type="any", mult="all")
```

50kb expression bin

```
library("ggsignif")
comp_gro50_red=comp_gro50[,c("expression", "combined_comp")]
par(mfrow=c(1,2), oma = c(0, 0, 2, 0), par(mar=c(4,4,0.1,0.1)))#bltr margins
boxplot(comp_gro50_red$expression~comp_gro50_red$combined_comp,
        outline=F,
        col=c("limegreen", "lightskyblue2", "maroon"),
        main="",
        ylab="expression",
        xlab="compartment")
# boogachacka, that's waht we were looking for
# plot log10values for better visualization
boxplot(log10(comp_gro50_red$expression)~comp_gro50_red$combined_comp,
        outline=F,
        col=c("limegreen", "lightskyblue2", "maroon"),
        main="",
        ylab="log10(expression)",
        xlab="compartment")
mtext("Nascent expression (50kb) in active and inactive compartments", outer = TRUE, cex = 1.2)
```



```
comp_gro50_red$col=c("limegreen","lightskyblue2","maroon")[comp_gro50_red$combined_comp]
my_comparisons <- list( c("AA", "AB"), c("AA", "BB"), c("AB", "BB"))

gp50 = ggplot(comp_gro50_red, aes(x=combined_comp, y=expression, fill=col)) + ylab("Expression") + xlab("Compartment Level") +
geom_violin(outlier.shape = NA) + scale_fill_manual(values=c("lightskyblue2", "limegreen", "maroon")) + theme_classic() + theme(legend.position="none")
stat_compare_means(comparisons = my_comparisons)

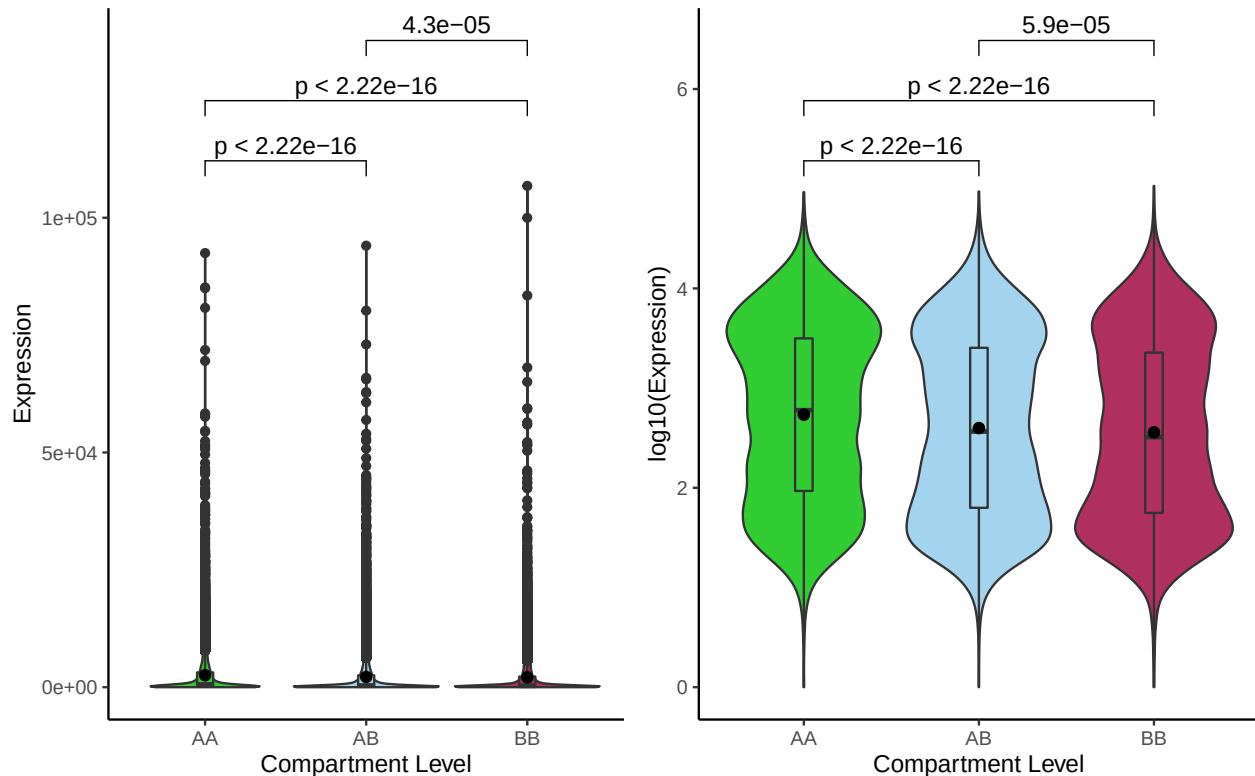
## Warning: Ignoring unknown parameters: outlier.shape

## Warning: `fun.y` is deprecated. Use `fun` instead.

gp50log = ggplot(comp_gro50_red, aes(x=combined_comp, y=log10(expression), fill=col)) + ylab("log10(Expression)") + xlab("Compartment Level") +
geom_violin() + scale_fill_manual(values=c("lightskyblue2", "limegreen", "maroon")) + theme_classic() + theme(legend.position="none")
stat_compare_means(comparisons = my_comparisons)

## Warning: `fun.y` is deprecated. Use `fun` instead.
grid.arrange(gp50, gp50log, ncol=2)

## Warning: Removed 22 rows containing non-finite values (stat_ydensity).
## Warning: Removed 22 rows containing non-finite values (stat_boxplot).
## Warning: Removed 22 rows containing non-finite values (stat_summary).
## Warning: Removed 22 rows containing non-finite values (stat_signif).
```



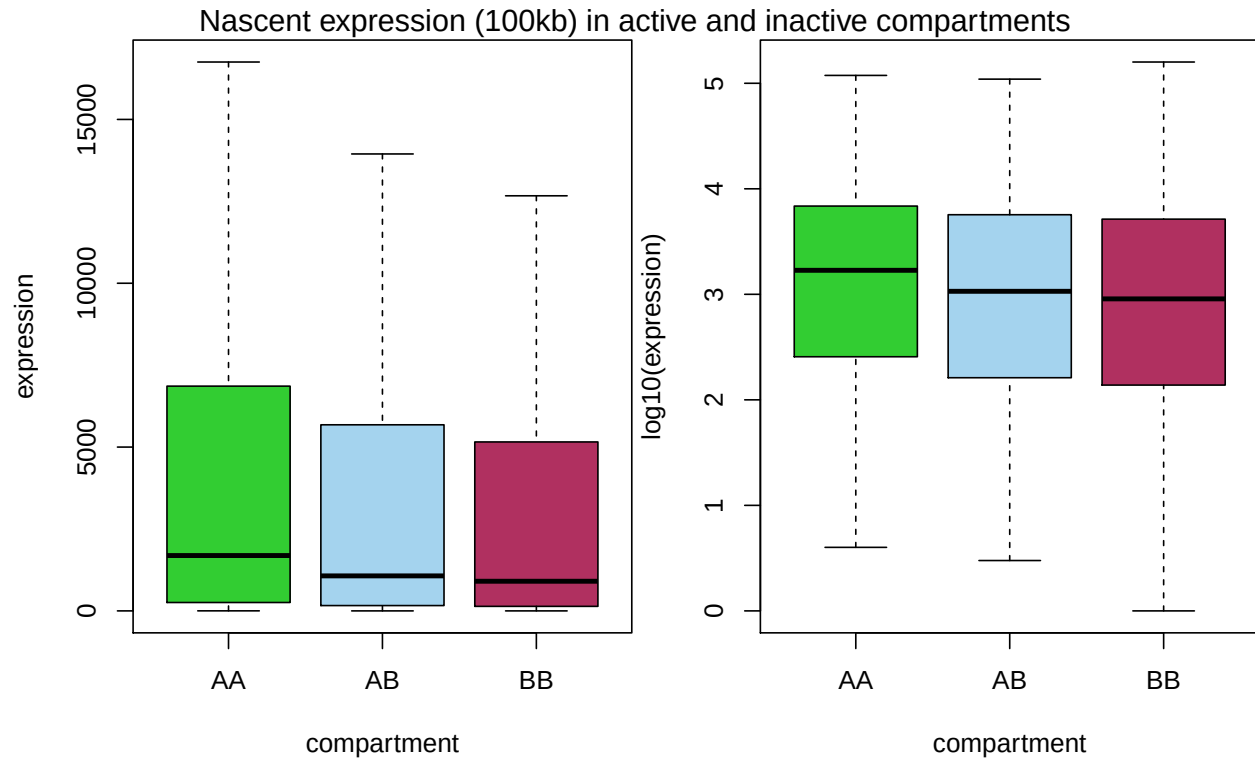
100kb expression bin

```
comp_gro100_red=comp_gro100[,c("expression","combined_comp")]
par(mfrow=c(1,2), oma = c(0, 0, 2, 0), par(mar=c(4,4,0.1,0.1)))#bltr margins
boxplot(comp_gro100_red$expression~comp_gro100_red$combined_comp,
        outline=F,
        col=c("limegreen","lightskyblue2","maroon"),
        main="",
        ylab="expression",
        xlab="compartment")
```

```

boxplot(log10(comp_gro100_red$expression)~comp_gro100_red$combined_comp,
        outline=F,
        col=c("limegreen","lightskyblue2","maroon"),
        main="",
        ylab="log10(expression)",
        xlab="compartment")
mtext("Nascent expression (100kb) in active and inactive compartments", outer = TRUE, cex = 1.2)

```



```

comp_gro100_red$col=c("limegreen","lightskyblue2","maroon")[comp_gro100_red$combined_comp]
my_comparisons <- list( c("AA", "AB"), c("AA", "BB"), c("AB", "BB"))

gp100 = ggplot(comp_gro100_red, aes(x=combined_comp, y=expression,fill=col)) + ylab("Expression") + xlab("Compartment Level") +
geom_violin(outlier.shape = NA) + scale_fill_manual(values=c("lightskyblue2","limegreen","maroon")) + theme_classic() + theme(legend.position="none")
stat_compare_means(comparisons = my_comparisons)

## Warning: Ignoring unknown parameters: outlier.shape
## Warning: `fun.y` is deprecated. Use `fun` instead.
gp100log = ggplot(comp_gro100_red, aes(x=combined_comp, y=log10(expression),fill=col)) + ylab("log10(Expression)") + xlab("Compartment Level") +
geom_violin(outlier.shape = NA) + scale_fill_manual(values=c("lightskyblue2","limegreen","maroon")) + theme_classic() + theme(legend.position="none")
stat_compare_means(comparisons = my_comparisons)

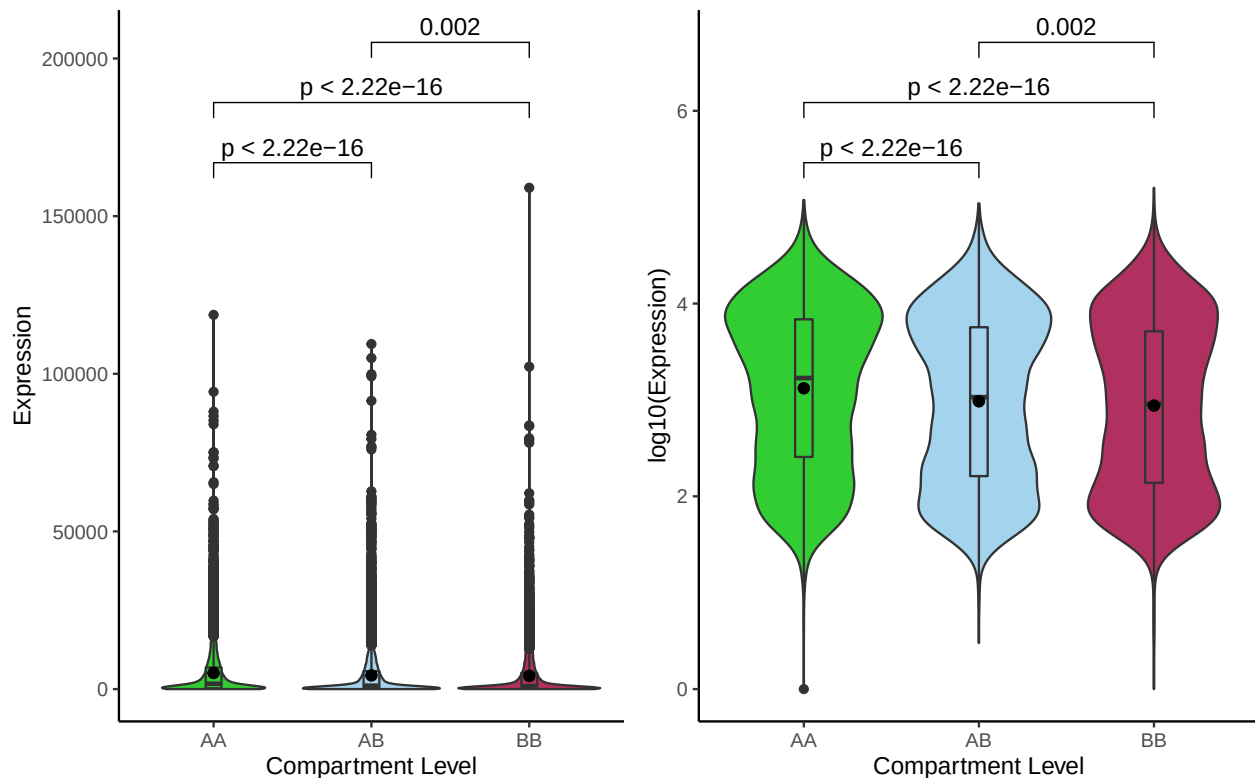
## Warning: `fun.y` is deprecated. Use `fun` instead.
grid.arrange(gp100, gp100log, ncol=2)

```

```

## Warning: Removed 2 rows containing non-finite values (stat_ydensity).
## Warning: Removed 2 rows containing non-finite values (stat_boxplot).
## Warning: Removed 2 rows containing non-finite values (stat_summary).
## Warning: Removed 2 rows containing non-finite values (stat_signif).

```



Additional Significance test...

...just to be sure the correct test was chosen, as we have confirmed before, that the expression data is not normally distributed. Thus, use Kruskal-Wallis test by rank to test if the observed differences are significant. Here is the thought process: We have 3 groups, all based on one single grouping variable. That means, we use either the one-way analysis of variance (ANOVA), also known as one-factor ANOVA, as it describes an extension of independent two-samples t-test for comparing means in a situation where there are more than two groups; or the Kruskal-Wallis test by rank, a non-parametric alternative to one-way ANOVA test, which extends the two-samples Wilcoxon test in the situation where there are more than two groups. It's recommended when the assumptions of one-way ANOVA test are not met. As we know that the normal distribution assumptions of ANOVA are not met in this case, I chose to use the Kruskal-Wallis test.

*# Kruskal-Wallis test is a non-parametric alternative to one-way ANOVA test,
which extends the two-samples Wilcoxon test in the situation where there are more
than two groups. It's recommended when the assumptions of one-way ANOVA test are not met.*

```
kruskal.test(expression ~ combined_comp, data = comp_gro100_red)
```

```
##
## Kruskal-Wallis rank sum test
##
## data: expression by combined_comp
## Kruskal-Wallis chi-squared = 170.39, df = 2, p-value < 2.2e-16
```

*# As the p-value is less than the significance level 0.05, we can conclude that
there are significant differences between the treatment groups.
From the output of the Kruskal-Wallis test, we know that there is a significant
difference between groups, but we don't know which pairs of groups are different.
It's possible to use the function pairwise.wilcox.test() to calculate pairwise
comparisons between group levels with corrections for multiple testing.*

```
pairwise.wilcox.test(comp_gro50_red$expression, comp_gro50_red$combined_comp, p.adjust.method = "BH")
```

```
##
## Pairwise comparisons using Wilcoxon rank sum test
##
## data: comp_gro50_red$expression and comp_gro50_red$combined_comp
```

```
##
##      AA      AB
## AB < 2e-16 -
## BB < 2e-16 4.3e-05
##
## P value adjustment method: BH
# those numbers are not identical to the calculations done using ggplot, but the result
# is the same, all comparisons are significant for the 50kb resolution.

pairwise.wilcox.test(comp_gro100_red$expression, comp_gro100_red$combined_comp,p.adjust.method = "BH")

##
## Pairwise comparisons using Wilcoxon rank sum test
##
## data: comp_gro100_red$expression and comp_gro100_red$combined_comp
##
##      AA      AB
## AB <2e-16 -
## BB <2e-16 0.002
##
## P value adjustment method: BH
# those numbers are not identical to the calculations done using ggplot, but the result
# is the same, all comparisons are significant for the 100kb resolution as well.

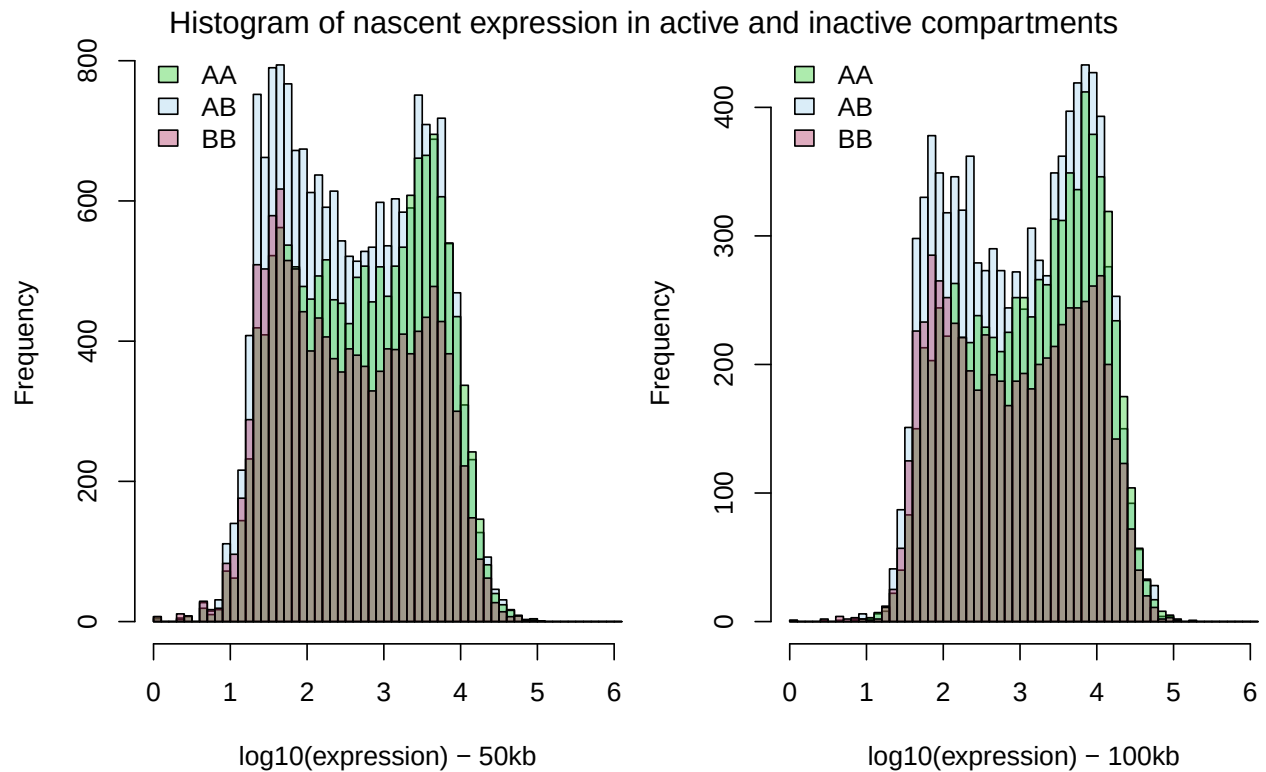
pairwise.wilcox.test(log10(comp_gro100_red$expression), comp_gro100_red$combined_comp,p.adjust.method = "BH")

##
## Pairwise comparisons using Wilcoxon rank sum test
##
## data: log10(comp_gro100_red$expression) and comp_gro100_red$combined_comp
##
##      AA      AB
## AB <2e-16 -
## BB <2e-16 0.002
##
## P value adjustment method: BH
# The results hold true with log transformation.
# The results are in line with the ggplot results.
```

Are the different expression distributions overlapping?

```
par(mfrow=c(1,2),oma = c(0, 0, 2, 0),par(mar=c(4,4,0.1,0.1)))#bltr margins
hist(log10(comp_gro50_red$expression[which(comp_gro50_red$combined_comp=="AB")] ),breaks = seq(0,6.1,0.1),col=adjustcolor("lightskyblue2",alpha.f=0.4),main="",
      xlab="log10(expression) - 50kb")
hist(log10(comp_gro50_red$expression[which(comp_gro50_red$combined_comp=="AA")] ),breaks = seq(0,6.1,0.1),col=adjustcolor("limegreen",alpha.f=0.4),main="",
      xlab="log10(expression) - 50kb")
hist(log10(comp_gro50_red$expression[which(comp_gro50_red$combined_comp=="BB")] ),breaks = seq(0,6.1,0.1),col=adjustcolor("maroon",alpha.f=0.4),main="",
      xlab="log10(expression) - 50kb")
legend("topleft",fill=adjustcolor(c("limegreen","lightskyblue2","maroon"),alpha.f=0.4),legend=c("AA","AB","BB"),bty="n")

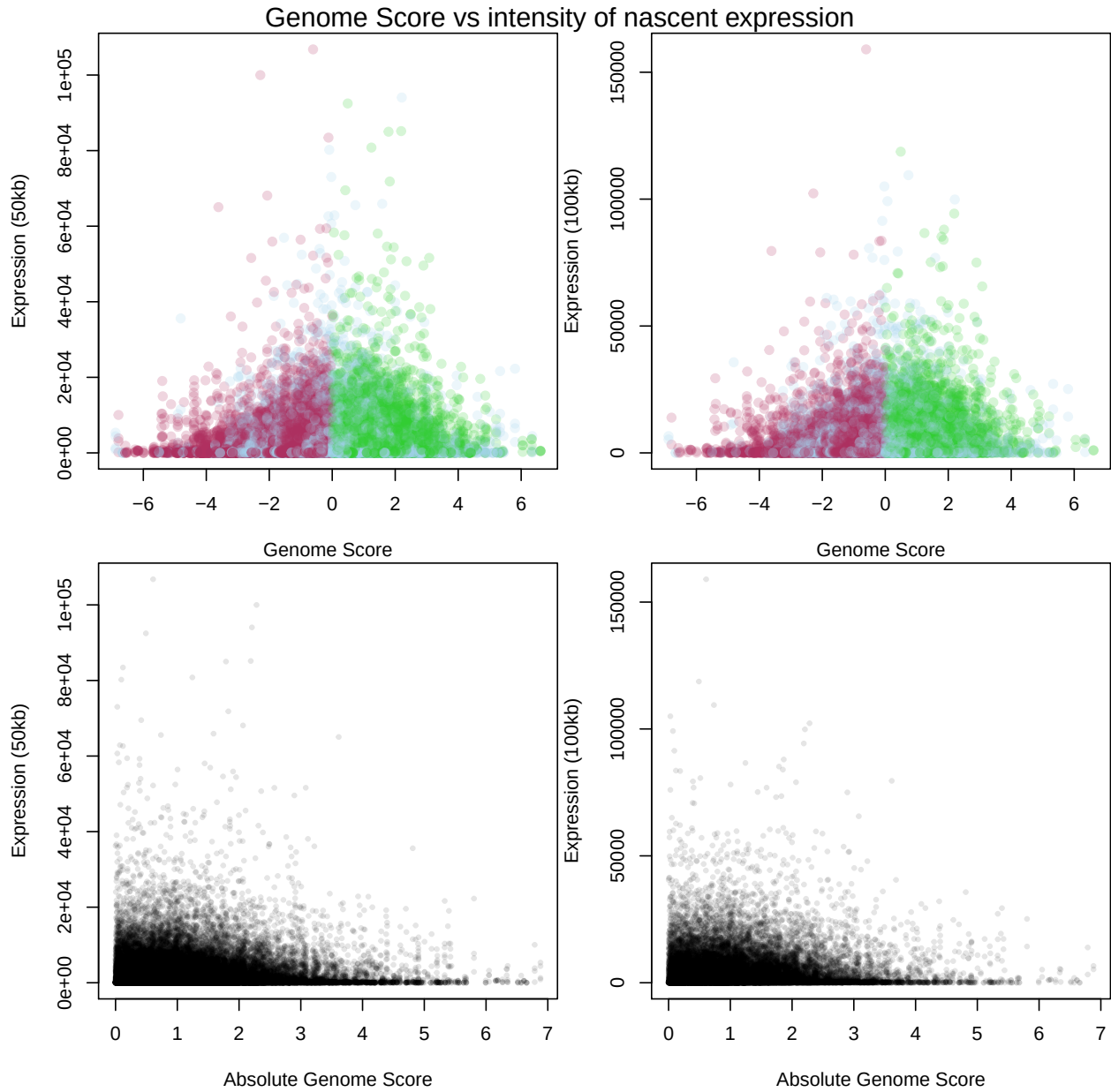
hist(log10(comp_gro100_red$expression[which(comp_gro100_red$combined_comp=="AB")] ),breaks = seq(0,6.1,0.1),col=adjustcolor("lightskyblue2",alpha.f=0.4),main="",
      xlab="log10(expression) - 100kb")
hist(log10(comp_gro100_red$expression[which(comp_gro100_red$combined_comp=="AA")] ),breaks = seq(0,6.1,0.1),col=adjustcolor("limegreen",alpha.f=0.4),main="",
      xlab="log10(expression) - 100kb")
hist(log10(comp_gro100_red$expression[which(comp_gro100_red$combined_comp=="BB")] ),breaks = seq(0,6.1,0.1),col=adjustcolor("maroon",alpha.f=0.4),main="",
      xlab="log10(expression) - 100kb")
legend("topleft",fill=adjustcolor(c("limegreen","lightskyblue2","maroon"),alpha.f=0.4),legend=c("AA","AB","BB"),bty="n")
mtext("Histogram of nascent expression in active and inactive compartments", outer = TRUE, cex = 1.2)
```



Do high expression values correlate with Eigenvalues of PC1?

```
par(mfrow=c(2,2),oma = c(0, 0, 2, 0),par(mar=c(4,4,0.1,0.1)))#bltr margins
plot(x=comp_gro50$Genome1_score1,
     y=comp_gro50$expression,pch = 19,
     col = adjustcolor(comp_gro50_red$col,alpha.f=0.2),
     xlab="Genome Score",
     ylab="Expression (50kb)")
plot(x=comp_gro100$Genome1_score1,
     y=comp_gro100$expression,pch = 19,
     col = adjustcolor(comp_gro100_red$col,alpha.f=0.2),
     xlab="Genome Score",
     ylab="Expression (100kb)")

plot(x=abs(comp_gro50$Genome1_score1),
     y=comp_gro50$expression,pch = 19,cex=0.5,
     col = adjustcolor("black",alpha.f=0.1),
     xlab="Absolute Genome Score",
     ylab="Expression (50kb)")
plot(x=abs(comp_gro100$Genome1_score1),
     y=comp_gro100$expression,pch = 19,cex=0.5,
     col = adjustcolor("black",alpha.f=0.1),
     xlab="Absolute Genome Score",
     ylab="Expression (100kb)")
mtext("Genome Score vs intensity of nascent expression", outer = TRUE, cex = 1.2)
```



Expression values are higher with lower Eigenvalues. This was unexpected to me, in case there would be a relationship, I expected strong negative values to correlate with low expression and high positive values to correlate with high expression.

Fazit

Here I could show, that the haplotype-specific H1 chromatin contact maps generated with the Co-phasing approach using hard distance cut offs between observed SNVs and reads, produce biologically meaningful results, as genomic regions active in both homologous chromosomes show significantly higher expression levels compared to regions which are active in one and inactive in the other homologous chromosome, while regions which are inactive in both show the lowest expression level.